

УДК Lorem ipsum dolor sit amet

Операционная семантика выражений в языке Go на языке ABML

Харьков А.А. (Студент ММФ НГУ)

Ануреев И.С. (Институт систем информатики СО РАН)

Lorem ipsum dolor sit amet

1. Введение

Lorem ipsum dolor sit amet

2. Ограничения фрагмента языка Go

Полное моделирование языка представляет собой задачу исключительной сложности. Современные языки программирования включают сотни синтаксических конструкций, множество тонких семантических правил и исторические особенности, накопившиеся за годы развития. Разумеется, зачастую достаточно знать лишь те конструкции языка программирования, что используются чаще всего. Ограничение исследуемых конструкций языка Go так же можно обосновать следующими соображениями:

1. Сокращение общего объёма кода способствует упрощению процесса выявления ошибок и неточностей в программной реализации;
2. Уменьшенный объём и структурированность кода облегчают анализ разработанных решений, позволяют чётко выделить ключевые закономерности, что, в свою очередь, повышает научную ценность проведённого исследования.

Поэтому в данной работе речь пойдёт именно про базовую часть языка Go, без использования каких-либо библиотек или пакетов.

Предварительная обработка исходного кода осуществляется с помощью лексического парсера, преобразующего сырой код в унифицированную форму. Данная операция сопоставима с этапами предварительной обработки, выполняемыми компилятором перед генерацией исполняемого кода. Далее опишем конкретные функции нашего лексического парсера:

1. Парсер явно указывает типы всех объектов. Это реализуемо потому, что язык Go является строго типизированным, значит на этапе компиляции уже известны типы всех объектов программы;
2. В языке Go, например, при декларации нескольких переменных в одном блоке, допускается опускать значения. В таких случаях парсер самостоятельно подставляет значение с предыдущей строки блока;
3. Все константные значения вычисляются на этапе компиляции программы. Поэтому парсер убирает все декларации констант, заменяя их имена соответствующими значениями. В том числе значение `iota`;
4. Некоторые способы создания пользовательских типов на самом деле являются обычным переименованием уже существующих. Поэтому на этапе компиляции производится подстановка имён типов;
5. Так же вместо функций и интерфейсов с шаблонными типами создаются экземпляры с необходимыми подставленными типами;
6. В языке Go доступна возможность краткого декларирования переменных `x := 1`, что с учётом замечаний выше очевидно приводится к полному декларированию переменных `var x int = 1`;
7. Импорт пакетов в код программы интерпретируется как декларативный механизм указания экспортируемых составляющих пакета - функций, переменных и других элементов - до старта выполнения программы. Доступ к этим элементам осуществляется по составным именам вида `<название пакета>.<название элемента>`.

3. Онтология фрагмента языка Go

Прежде чем разбираться с моделированием действий различных программ или их составляющих, необходимо предварительно создать модель для элементов, составляющих язык Go, которые присутствуют в коде программы: переменные, функции, операторы, декларации и т.д. Для создания онтологии фрагмента языка Go, на языке ABML будут созданы типы объектов, соответствующие некоторым объектам фрагмента языка Go.

Классификация и порядок изложения опираются на официальную спецификацию языка Go. Большая часть названий объектов, моделирующихся в этом разделе, будет совпадать с их названиями в официальной спецификации для удобства восприятия и облегчения поиска дополнительной информации по каждому из них. Однако некоторые сущно-

сти были признаны излишними с точки зрения моделирования их функциональности, а некоторые имена недостаточно отражающими суть описываемого объекта. Это вызвало некоторую разницу в именовании нескольких объектов в построенной модели.

3.1. Имена

Ниже создаются модели для типа, описывающего имена объектов в языке Go. Отмечу, что для полного понимания изложенного здесь материала необходимо понимание основных команд языка ABML, про который можно подробнее узнать из статей в списке литературы. Однако в данном подразделе будет дано небольшое пояснение по написанному коду на языке ABML. Ключевое слово `typedef` означает, что данная инструкция моделирует некоторый класс объектов. Следующее за ним слово `"identifier"` указывает на название создаваемого класса. Конструкция `(uniont ...)` показывает объекты, которые могут быть причислены к данному классу. Причём важно отметить, что это определяет необходимые свойства для установления принадлежности некоторого объекта создаваемому классу, но не достаточные. То есть на данный момент при моделировании таких классов зачастую допускается большее число элементов, чем реально будет моделироваться данным классом.

Начнём с введения типов, моделирующих имена различных объектов программы.

```
1 (typedef "identifier" (uniont string))
2 (typedef "variable name" "identifier")
3 (typedef "field name" "identifier")
4 (typedef "function name" "identifier")
5 (typedef "method name" "identifier")
6 (typedef "interface name" "identifier")
7 (typedef "label name" "identifier")
8 (typedef "type name" "identifier")
9 (typedef "package name" "identifier")
```

Тип `"identifier"` описывает общее понятие имени объекта в программе. Остальные типы показывают отношение имени к более конкретным объектам языка Go.

Таким образом, `"variable name"` используется для имён переменных, `"field name"` - для имён полей в структурах, тип `"function name"` - для имён функций, тип `"method name"` - для имён методов, тип `"interface name"` - для имён интерфейсов, тип `"label`

`name`" - для имён меток, тип `"type name"` - для имён типов, которые не получилось подставить парсером, тип `"package name"` - для имён пакетов или внешних файлов.

3.2. Значения и память

В данном подразделе создаются типы для моделирования типизированных ячеек памяти и значений, которые могут в них храниться. Стоит отметить, что в силу типизированности языка Go, возможно явно указать тип каждого значения.

Начнём с типизированных ячеек памяти и общего понятия возможного значения в языке Go.

```
1 (mot "location" :at "type" "type" :at "value" "Go value")
2 (typedef "Go value" (uniont "literal" "location"))
```

добавить пояснение для type value value обозначает значение, хранимое в памяти

Далее моделируются типы для постоянных значений, в том числе комплексных чисел.

Такие значения явно вычисляются на этапе компиляции.

```
1 (typedef "constant" (uniont "true" "false" int real "complex
   constant" string symbol))
2 (mot "complex constant" :at "re" real :at "im" real)
```

Дальше речь пойдёт о литералах, которые являются значениями в том числе для составных объектов.

```
1 (typedef "literal" (uniont "constant" "composite lit" "function
   lit" "channel lit"))
2 (typedef "composite literal" (uniont "array lit" "slice lit"
   "struct lit" "map lit"))
3 (mot "array lit" :at "type" "array type" :at "value" (listt "Go
   value"))
4 (mot "slice lit" :at "type" "slice type" :at "value" (listt "Go
   value"))
5 (mot "struct lit" :at "type" "struct type" :at "value" (cot
   :amap "field name" "Go value"))
6 (mot "map lit" :at "type" "map type" :at "value" (cot :amap
   "identifier" "Go value"))
```

Тип `"literal"` определяет понятие литерала в общем.

Тип `"composite literal"` определяет понятие составного литерала.

Типы `"array lit"`, `"slice lit"`, `"struct lit"`, `"map lit"` моделируют объекты, представляющие собой значений для массивов, срезов, структур и карт соответственно.

В следующем блоке описывается тип функционального литерала и связанные с ним.

```
1 (mot "function lit" :at "signature" "signature" :at "body"
   "block") ; func(a int) bool { return a < 0 }
2 (mot "signature" :at "parameters" (listt "parameter decl") :at
   "variadic parameter" "variadic decl" :at "result" "function
   result")
3 (mot "parameter decl" :at "name" "identifier" :at "type" "type")
4 (mot "variadic decl" :at "name" "identifier" :at "type" "type")
   ; func (n ...int){}(1, 2, 3)
5 (mot "function result" :at "result" (uniont (listt "type")
   (listt "parameter decl")))
```

Тип `"function lit"` моделирует объекты, представляющие собой функции. Причём исходная инструкция в исходном коде не декларирует функцию, а создаёт функциональный объект, без привязки к конкретному имени.

Тип `"signature"` представляет собой сигнатуры функций - список параметров функции и её результат, выходные значения.

Тип `"parameter decl"` описывает каждый параметр в объявлении функции.

Тип `"variadic decl"` используется для получения возможности создавать функции с произвольным количеством аргументов. В исходном коде на языке Go перед типом соответствующего аргумента ставится троеточие, например, `func(n ...int)`.

Наконец опишем структуру, позволяющую моделировать значения каналов.

```
1 (mot "channel lit" :at "type" "channel type" :at "cap" nat :at
   "buffer" (listt "Go value"))
```

Тип `"channel literal"` позволяет моделировать взаимодействия каналов в программе.

3.3. Типы данных

В данном подразделе моделируются типы данных, большая часть которых должна быть интуитивно понятна из названий. Типы можно разделить на базовые - представляющие некоторые числа, символы или строки, и составные типы данных - состоящие зачастую совокупность базовых или моделирующих более сложные концепции хранения данных.

Сперва рассмотрим базовые типы данных.

```
1 (typedef "type" (uniont "base type" "composite type"))
2 (typedef "base type" (uniont "boolean type" "numeric type" "rune
   type" "string type"))
3 (typedef "boolean type" (enumt "bool"))
4 (typedef "numeric type" (uniont "int type" "float type" "complex
   type"))
5 (typedef "int type" (uniont "int" "int8" "int16" "int32" "int64"
   "uint" "uint8" "uint16" "uint32" "uint64" "uintptr" "byte"))
6 (typedef "float type" (enumt "float32" "float64"))
7 (typedef "complex type" (enumt "complex64" "complex128"))
8 (typedef "rune type" (enumt "rune"))
9 (typedef "string type" (enumt "string"))
```

Стоит отметить, что строки выше можно разделить на два вида типов.

Первые объединяют существующие типы данных общим понятием. К таким типам можно отнести следующие: тип `"type"` моделирует общее понятие типа данных, `"base type"` представляет собой простейшие типы данных (логический, числовой, строковый), `"numeric type"` объединяет числовые типы данных, `"int type"` - целочисленные типы данных.

Второй вид типов характеризуется явным перечислением входящих в него типов: `"boolean type"` моделирует логический тип данных, `"float type"` - вещественные числа, `"complex type"` - комплексные числа, имеющие возможность описать миную часть числа, `"rune type"` моделирует тип, элементами которого являются символы, в том числе спецсимволы, например к этому типу относятся `'a' | 't' | 'U00101234'`, и, наконец, последний тип `"string type"` представляет собой тип строк.

Далее перейдём к рассмотрению составных типов данных. Сначала явно перечислим, что понимается под понятием составного типа данных `"composite type"`.

```
1 (typedef "composite type" (uniont "array type" "slice type"
    "struct type" "pointer type" "function type" "interface type"
    "map type" "channel type"))
```

Начнём с простейших из них.

```
1 (mot "array type" :at "len" nat :at "element type" "type")
2 (mot "slice type" :at "element type" "type") ; var s []int
3 (mot "struct type" :at "fields" (cot :amap "field name" "type"))
4 (mot "pointer type" :at "type" "type")
```

Тип `"array type"` содержит типы, описывающие списки - структуры данных, которые можно описать как упорядоченный набор элементов одного типа. Стоит явно отметить, что типом элементов массива является именно тип составляющих его элементов. Например, `var a [2][2]int` создаёт массив, который может быть представлен в виде квадратной матрицы размера 2, и типом элементов в данном примере является `[2]int` - массив длины 2 из целых чисел. Тип срезов `"slice type"` содержит типы, описывающие срезы. Срез - структура данных, которую можно представить в виде ссылки на последовательную часть исходного массива.

Тип `"struct type"` содержит типы, позволяет создавать новые объекты, состоит из множества полей. Каждое поле структуры характеризуется именем и типом, аналогично обычной переменной.

Тип `"pointer type"` описывает типы ссылок на исходные объекты.

Далее речь пойдёт о функциях.

```
1 (mot "function type" :at "type signature" "type signature")
2 (mot "type signature" :at "types" (listt "type") :at "variadic
    type" "type" :at "result" (listt "type"))
```

Тип `"function type"` описывает типы функций, который характеризуется своей сигатурой типов `"type signature"`, которая в свою очередь является совокупностью типов параметров `"types"` и `"variadic type"`, и типом результата функции `"result"`. Тип `"variadic type"` обозначает неопределённое количество типов, на языке Go такой параметр можно поставить только последним, обозначается троеточием перед типом элементов.

Теперь несколько примеров для лучшего понимания связи синтаксиса типа функции в языке Go с моделируемым на языке ABML. Список параметров функции указывается в круглых скобках после ключевого слова `func`, результат функции указывается после закрытия круглых скобок, т.е. `func("types", "variadic type")"result"`. Например, `func(int, int, ...string)bool` обозначает тип функций, возвращающих логическое значение, и принимающих первыми двумя параметрами целочисленные значения, а последующими возможно передать произвольное количество строк.

Далее рассмотрим моделирование типа интерфейсов.

```
1 (not "interface type" :at "elements" (cot :amap "method name"
    "signature"))
```

Этот тип представляет собой интерфейсы с различными методами. Ниже приведу пример инициации интерфейса на языке Go.

```
1 type A interface {
2     f(n int)          // "method name" = 'f', "signature" = '(n int)'
3 }
```

Тип интерфейс `"interface type"` представим как множество, элементы которого можно назвать функциями без указания реализации. Синтаксически интерфейс состоит из названий функций и их сигнатур: параметры функции и возвращаемое значение.

Тип сигнатура `"signature"` можно назвать конкретизированием сигнатуры типов `"type signature"`, так как сигнатура содержит определённые имена параметров.

Осталось определить ещё несколько типов данных.

```
1 (not "map type" :at "key type" "type" :at "element type" "type")
2 (not "channel type" :at "direction" "direction" :at "type"
    "type")
3 (typedef "direction" (enumt "bidirectional" "send" "receive"))
```

Тип словарь (карта) `"map type"` является реализацией хэш-таблицы. Приведём пример такого типа на языке Go: `map[string]int`. В данном примере ключи таблицы имеют строковый тип, а значениями являются целые числа.

Тип канал `"channel type"` используется преимущественно в многопоточных процессах. Каналы можно представить как переменную с дополнительной особенностью - при считыва-

нии значения "пустой" ячейки происходит блокирование процесса до тех пор, пока другой процесс не положит в данную ячейку какое-либо значение. Причём каждый канал имеет своё направление "**direction**" - некоторые каналы предназначены только для записи или только для получения данных из него, или всё вместе. Наглядный пример использования каналов на языке Go будет приведён ниже, в конце раздела выражений.

3.4. Блоки

Этот подраздел посвящён только одному типу блоков "**block**", в том числе по причине обособленности с точки зрения описания функциональности.

```
1 (mot "block"
2   :at "statements" (listt "statement")
3   ; semantic attributes
4   :at "variable location" (cot :amap "variable name"
5     "location")
6   :at "label position" (cot :amap "label" nat)
7 )
```

Атрибут "**statements**" представляет собой список команд из блока, переведённых в модельное представление.

Следующие атрибуты не являются необходимыми для представления типа "**block**", однако упрощают реализацию действий некоторых команд, включающих в себя взаимодействие с блоками. Поэтому допустимо называть следующие атрибуты семантическими.

Для каждой переменной, инициализируемой в этом блоке, сохраняется её ячейка памяти до начала обработки блока. Если в блоке будет инициализирована переменная с таким же именем, так называемая локальная переменная, то в результате под старым именем будет использоваться новая переменная, а после завершения обработки блока необходимо будет вернуть старую переменную - соотнести имени переменной её ячейку памяти до начала обработки блока. Если переменной раньше не существовало, значение по умолчанию устанавливается **nil**

Атрибут "**label position**" является отношением, сопоставляющим каждой метке её позицию в этом блоке.

3.5. Декларации

Настало время определиться с декларированием различных объектов. Начнём с общего понятия декларации объекта `"declaration"`.

```
1 (typedef "declaration" (uniont "type decl" "var decl" "function
    decl" "method decl" "interface decl"))
```

Далее опишем декларирование типов `"type decl"`

```
1 (mot "type decl" :at "name" "type name" :at "type" "type")
```

Сразу же приведёт пример соответствующей конструкции на языке Go:

```
1 type Point struct{ x bool }
```

В данном примере объявляется тип `Point`, элементами которого являются структуры с одним полем `x` типа `bool`. Причём компилятор не производит подстановку определённых таким образом типов на этапе компиляции программы.

Теперь опишем декларацию переменных, разделяя её на 2 типа: полная декларация `"var decl"` с явным указанием типа переменной, и короткая декларация `"short var decl"`, тип которой устанавливается по соответствующему начальному значению при декларации.

```
1 (mot "var decl" :amap "variable name" "type & value")
2 (mot "type & value" :at "value" "expression" :at "type" "type")
```

Здесь можно привести пример декларации переменной на языке Go: `var x int = 1`. Так же стоит отметить, что тип `"type & value"` не является синтаксически уникальным, однако отличается от ранее заданного типа `"location"` с точки зрения семантики языка.

Теперь, перейдём к декларированию функциональных объектов: функций `"function decl"` и методов `"method decl"`.

```
1 (mot "function decl" :at "name" "function name" :at "signature"
    "signature" :at "body" "block")
```

Далее приведём пример декларации функции на языке Go:

```
1 func add(p *Point, q *Point) {
2     p.x += q.x
3 }
```

Соответственно декларация методов.

```
1 (mot "method decl" :at "receiver" "parameter decl" :at "name"
   "method name" :at "signature" "signature" :at "body" "block")
```

Далее приведём пример декларации метода на языке Go:

```
1 func (p *Point) add(q *Point) {
2     p.x += q.x
3 }
```

Не трудно заметить, что функции и методы очень похожи между собой.

Наконец, данный подраздел заканчивает декларирование интерфейсов.

```
1 (mot "interface decl" :at "name" "interface name" :at "type"
   "interface type")
```

3.6. Выражения

Данный раздел посвящён моделированию выражений - некоторых команд, что после исполнения возвращают некоторое значение. Начнём с перечисления типов, являющихся выражениями "expression".

```
1 (typedef "expression" (uniont "literal" "conversion" "method
   expr" "selector expr" "index expr" "slice expr" "type
   assertion" "function call" "unary expression" "binary
   expression"))
```

Теперь приступим к определению перечисленных выше типов.

```
1 (mot "conversion" :at "type" "type" :at "expression"
   "expression")
2 (mot "method expression" :at "receiver type" "type" :at "name"
   "method name")
```

Элементы типа "conversion" позволяют выполнять приведение типов. К данной конструкции можно привести следующий пример приведения целочисленного значения **2** к типу `float64`: `float64(2) == 2.0`.

Элементы типа `"method expression"` позволяют создавать функции по методам. Такие функции получаются перестановкой получателя метода на место первого параметра функции, таким образом можно говорить об эквивалентности метода и порождаемой им функции.

Далее опишем выражения выбора поля структуры и метода по экземпляру структуры.

```
1 (typedef "selector expression" (uniont "selector struct"
    "selector method"))
2 (mot "selector struct" :at "struct" "expression" :at "field"
    "field name")
3 (mot "selector method" :at "method" "expression" :at "method"
    "method name")
```

Для выражения выбора поля структуры можем использовать следующий пример на языке Go: `person.Age`. Для выражения выбора метода по экземпляру структуры можем привести следующее: `file.Close()`.

Следующим опишем похожий на 2 предыдущих тип, но в данном случае выбирается элемент индексированной структуры данных.

```
1 (mot "index expr" :at "arr" "expression" :at "index"
    "expression")
```

Данное выражение применимо к следующим типам данных: массивы, срезы, карты и строки.

Ранее упоминалось, что срез является ссылкой на исходный массив, теперь введём тип для выражений, позволяющих создавать срезы по уже существующему массиву.

```
1 (mot "slice expr" :at "arr" "expression" :at "low" "expression"
    :at "high" "expression" :at "max" "expression")
```

Это выражение содержит не более трёх аргументов, последний из которых указывать не обязательно, поскольку он по умолчанию устанавливается равным разности длины исходного массива (или строки - массива символов) и параметра `low`. Параметр `low` является индекс в исходном массиве, от которого срез начинается, включительно. Параметр `high` соответственно является конечным индексом, но уже исключительно. Параметр `max` устанавливает максимально возможную ёмкость среза, таким образом по умолчанию срез

может расти вплоть до последнего элемента массива.

Следующая конструкция используется для приведения интерфейсных типов данных, возвращает получилось ли выполнить соответствующее приведение.

```
1 (mot "type assertion" :at "interface" "expression" :at "type"
   "type")
```

В качестве примера к последней инструкции можно привести следующее:

```
1 var i interface{} = 3
2 num := i.(int) // Успех: num = 3 (тип int)
```

```
1 (mot "function call" :at "function" "expression" :at "arguments"
   (listt "expression")) ; min(1, 2)
2 (mot "<-1" :at 1 "expression") ; Return the value from the
   channel 1
```

Далее описано создание типов для унарных алгебраических выражений из языка Go.

```
1 (typedef "unary expression" (uniont "+1" "-1" "^1" "!1" "*1"
   "&1"))
2 (mot "+1" :at 1 "expression") ; +x == x
3 (mot "-1" :at 1 "expression") ; -x == -1 * x
4 (mot "^1" :at 1 "expression") ; ^10(2) == 01(2)
5 (mot "!1" :at 1 "expression") ; !true == false
6 (mot "*1" :at 1 "expression")
7 (mot "&1" :at 1 "expression")
```

Далее описываются типы для бинарных алгебраических выражений из языка Go.

```
1 (typedef "binary expression" (uniont "1||2" "1&&2" "1*2" "1/2"
   "1%2" "1<<2" "1>>2" "1&2" "1&^2" "1+2" "1-2" "1|2" "1^2"
   "1=2" "1!2" "1<2" "1<2" "1>2" "1>2"))
2 (mot "1||2" :at 1 "expression" :at 2 "expression")
3 (mot "1&&2" :at 1 "expression" :at 2 "expression")
4 (mot "1*2" :at 1 "expression" :at 2 "expression")
5 (mot "1/2" :at 1 "expression" :at 2 "expression")
```

```

6 (mot "1%2" :at 1 "expression" :at 2 "expression")
7 (mot "1<<2" :at 1 "expression" :at 2 "expression")
8 (mot "1>>2" :at 1 "expression" :at 2 "expression")
9 (mot "1&2" :at 1 "expression" :at 2 "expression")
10 (mot "1&^2" :at 1 "expression" :at 2 "expression") ; 1 &^ 2 == 1
    & (~2)
11 (mot "1+2" :at 1 "expression" :at 2 "expression")
12 (mot "1-2" :at 1 "expression" :at 2 "expression")
13 (mot "1|2" :at 1 "expression" :at 2 "expression")
14 (mot "1^2" :at 1 "expression" :at 2 "expression")
15 (mot "1==2" :at 1 "expression" :at 2 "expression")
16 (mot "1!=2" :at 1 "expression" :at 2 "expression")
17 (mot "1<2" :at 1 "expression" :at 2 "expression")
18 (mot "1<=2" :at 1 "expression" :at 2 "expression")
19 (mot "1>2" :at 1 "expression" :at 2 "expression")
20 (mot "1>=2" :at 1 "expression" :at 2 "expression")

```

Представленные выше алгебраические выражения имеют привычный, по известным языкам программирования, синтаксис. Так же описание конкретного значения для каждого такого выражения по отдельности не представляет собой смысла для данной работы.

3.7. Операторы

```

1 (typedef "statement" (uniont "declaration" "labeled stmt"
    "simple stmt" "go stmt" "return stmt" "break stmt" "continue
    stmt" "goto stmt" "fallthrough stmt" "block" "if stmt"
    "switch stmt" "select stmt" "for stmt" "defer stmt"))

```

```

1 (mot "label stmt" :at "label" "label" :at "statement"
    "statement")
2 (mot "label" :at "name" "identifier") ; "label": "statement"
3 (typedef "simple stmt" (uniont "empty stmt" "expression stmt"
    "send stmt" "1++" "1--" "assignment stmt"))
4 (typedef "empty stmt") ; The empty statement does nothing

```

```

5 (mot "expression stmt" :at "expression" "expression")
6 ; h(x+y)
7 ; f.Close()
8 (mot "send stmt" :at "channel" "expression" :at "message"
   "expression")
9 (mot "1++ stmt" :at "1" "expression" ; x += 1
10 (mot "1-- stmt" :at "1" "expression" ; x -= 1

```

```

1 (typedef "assignment stmt" (uniont "1=2" "1+=2" "1-=2" "1|=2"
   "1^=2" "1*=2" "1/=2" "1%=2" "1<=2" "1>=2" "1&=2" "1&^=2"))
2 (mot "1=2" :at 1 (listt "expression") :at 2 (listt "expression"))
3 (mot "1+=2" :at 1 (listt "expression") :at 2 (listt
   "expression"))
4 (mot "1-=2" :at 1 (listt "expression") :at 2 (listt
   "expression"))
5 (mot "1|=2" :at 1 (listt "expression") :at 2 (listt
   "expression"))
6 (mot "1^=2" :at 1 (listt "expression") :at 2 (listt
   "expression"))
7 (mot "1*=2" :at 1 (listt "expression") :at 2 (listt
   "expression"))
8 (mot "1/=2" :at 1 (listt "expression") :at 2 (listt
   "expression"))
9 (mot "1%=2" :at 1 (listt "expression") :at 2 (listt
   "expression"))
10 (mot "1<=2" :at 1 (listt "expression") :at 2 (listt
   "expression"))
11 (mot "1>=2" :at 1 (listt "expression") :at 2 (listt
   "expression"))
12 (mot "1&=2" :at 1 (listt "expression") :at 2 (listt
   "expression"))
13 (mot "1&^=2" :at 1 (listt "expression") :at 2 (listt

```

```
"expression"))
```

```
1 (not "if stmt" :at "init" "simple stmt" :at "condition"
   "expression" :at "then" "block" :at "else" (uniont "if
   statement" "block"))
2 ; if x := f(); x < y {
3 ;     return x
4 ; } else if x > z {
5 ;     return z
6 ; }
```

Оператор switch

```
1 (typedef "switch stmt" (uniont "expr switch stmt" "type switch
   stmt"))
2 (not "expr switch stmt" :at "init" "simple stmt" :at
   "controlling expression" "expression" :at "cases" (listt
   "expr case clause"))
3 (not "expr case clause" :at "cases" (uniont (listt "expression")
   "default") :at "statements" (listt "statement"))
4 (cot "default")
5 ; switch a {
6 ; case 0, 1, 2: f()
7 ; default: g()
8 ; }
9 (not "type switch stmt" :at "init" "simple stmt" :at "guard"
   "type switch guard" :at "cases" (listt "type case clause"))
10 (not "type switch guard" :at "name" "identifier" :at "variable"
    "primary expression")
11 (not "type case clause" :at "cases" (uniont (listt "type")
    "default") :at "statements" (listt "statement"))
12 ; switch t := x.(type) {           // type checker
13 ; case nil:
14 ;     fmt.Println("x is nil")
```

```

15 ; default:
16 ;     fmt.Println("don't know the type")
17 ;;for statement
18 (typedef "for statement" (uniont "for stmt with condition
    clause" "for stmt with range clause"))
19 (mot "for stmt with condition clause" :at "init" "simple stmt"
    :at "condition" "expression" :at "post" "simple stmt" :at
    "body" "block")
20 (mot "for stmt with range clause" :at "names" (uniont (listt
    "expression") (listt "identifier"))) :at "expression"
    "expression" :at "body" "block")
21 ; for x < y { a *= 2 }                // the usual while loop
22 ; for i := 0; i < 10; i++ { f(i) }    // the usual for loop
23 ; a := [5]int{1, 2, 3, 4, 5}
24 ; for i, v := range a {
25 ;     fmt.Println(i, v)                // i a[i]
26 ; }
27 (mot "go stmt" :at "body" "expression") ; It starts the
    execution of a function call as an independent concurrent
    thread of control
28 ; go Server()
29 ; go func(ch chan<- bool) { for { sleep(10); ch <- true }} (c)
30 (mot "select stmt" :at "statement" (listt "common clause"))
31 (mot "common clause" :at "case" "common case" :at "statements"
    (listt "statement"))
32 (typedef "common case" (uniont "send stmt" "recv = stmt"
    "receive := stmt" "default"))
33 (mot "receive = stmt" :at "names" (listt "expression") : at
    "expression" "expression")
34 (mot "receive := stmt" :at "names" (listt "identifier") : at
    "expression" "expression")
35 ; func f1(c1 chan int) {

```

```
36 ;      a := <-c1
37 ;      c1 <- a
38 ; }
39 ; func f2(c2 chan int) {
40 ;      b := <-c2
41 ;      c2 <- b
42 ; }
43 ; c1, c2 := make(chan int), make(chan int)
44 ; go f1(c1)
45 ; go f2(c2)
46 ; c1 <- 3
47 ; c2 <- 4
48 ; select {
49 ; case a := <-c1:
50 ;     fmt.Println(1, a)
51 ; case b := <-c2:
52 ;     fmt.Println(2, b)
53 ; }
54 (mot "return" :at "expressions" (listt "expression"))
55 (mot "break " :at "label" "label")
56 ; OuterLoop:
57 ;     for {
58 ;         for {
59 ;             break OuterLoop
60 ;         }
61 ;     }
62 (mot "continue" :at "label" "label")
63 (mot "goto" :at "label" "label")
64 (typedef "fallthrough" (enumt "fallthrough"))
65 ; switch {                                // prints "true & false"
66 ;     case true:
67 ;         fmt.Print("true & ")
```

```
68 ;      fallthrough          // It transfers control to the
    next case clause in a switch
69 ;      case false:
70 ;      fmt.Print("false")
71 ; }
72 (not "defer stmt" :at "expression" "expression") ; A "defer"
    statement invokes a function whose execution is deferred to
    the moment the surrounding function returns
73 ; for i := 0; i <= 3; i++ {    // prints 3 2 1 0
74 ;      defer fmt.Print(i)
75 ; }
```

4. Операционная семантика языка Go